

David Marji, Brian Morava, Maxim Miladinov-Genov, Alexander Wyrstiuk
261086407, 261032388, 260989667, 261022984

Group P- 2

Mona ElSaadawy, Bettina Kemme

Database Systems

03 February 2026

Comp421 P1

1) Requirements Analysis

1.1) Introduction

a) Purpose

This database application is designed to support project management systems for a specific organization (this is not a general app, this is for one organization). It stores and manages information about employees, their role (standard, manager, director, and/or supervisor). It tracks the different teams, projects, and parts of the project, such as components, objectives, and tasks. The application additionally tracks meetings, who attends them, etc. The main goal is to ensure accountability (who supervises/manages what, who did what task, etc.) and the traceability of work (project → component → objective → task).

b) Scope and special requirements

The scope is limited to the organization's internal structure and workflow:

- Employees exist in exactly one role: Standard (standard employee), Manager, Director, or Supervisor.
- Projects are managed at the director level, decomposed into components and subcomponents.
- Components are supervised; objectives are distributed to teams; tasks are assigned to standard employees.
- Meetings exist as coordination artifacts with employee participation.

c) Terminology

- Standard / standard employee
 - A regular employee with no special managerial roles. Can be anything from a junior to a senior position.
- Component
 - A subunit of a project that can have spinoff subcomponents.

- Objective
 - A goal associated with a project.
- Task
 - An individual unit of work is assumed to be small enough to be completed by one employee.

1.2 Data Requirements

a) Entities and their attributes

Important note: We intentionally decided to implement all creation logic/constraints in the application layer to avoid storing unimportant information about who created what project, task, team, etc. So in the following requirements, any mention of “X can create Y” will be implemented in the application layer.

1. Employee
 - a. Stores employment and identity information for all staff.
 - b. Attributes:
 - i. emp_id (unique identifier)
 - ii. name
 - iii. email
 - iv. password
 - v. position_title (used to differentiate between positions at the same level of hierarchy like hiring manager, backend team lead, etc.)
 - vi. start_date (date employee was hired)
2. Standard (Standard Employee)
 - a. A subclass of Employees, representing non-managerial positions, can be assigned Tasks by a Manager and can belong to a Team.
 - b. Inherits all Employee attributes and has no extra attributes.
3. Manager
 - a. A subclass of Employees, representing low-level managerial positions, can create and assign Tasks to Standard Employees in a team they manage. Manages Teams.
 - b. Inherits all Employee attributes and has no extra attributes.
4. Supervisor
 - a. Subclass of Employee, represents employees who supervise Components, and create and distribute Objectives to Teams. Can create Teams for their department and assign Managers to manage them.
 - b. Inherits all Employee attributes and has no extra attributes.
5. Director
 - a. Subclass of Employee, represents employees who run and create Projects, can also create Components of a Project. A Director runs at most one project.

- b. Inherits all Employee attributes and no extra attributes.
- 6. Team
 - a. A unit within the organization. Standard employees belong to Teams and Managers manage Teams. Teams have Objectives distributed to them by Supervisors
 - b. Attributes:
 - i. id (unique identifier)
 - ii. Department (string used to show what department the team belongs to)
- 7. Project
 - a. A major initiative executed by the organization. Created by Directors, broken down into Components by Supervisors or Directors. Must have at least one Director.
 - b. Attributes:
 - i. codename (unique identifier)
 - ii. charter
 - iii. start_date (creation date of an instance of Project)
 - iv. deadline_date (deadline for the project)
- 8. Components
 - a. Subdivision of Project, only created by Directors. Can have spinoff Components (also created by Directors).
 - b. Attributes:
 - i. name (partial key)
- 9. Objectives
 - a. Weak entity set. Goals tied to Components created by Supervisors.
 - b. Attributes:
 - i. obj_id (partial key)
 - ii. goal_name
 - iii. Description
- 10. Tasks
 - a. Weak entity set. Units of work are assumed to be individually small enough to be assigned to and completed by a single Standard Employee. Created and Assigned by Managers. Identified only within an objective, for a component of a project.
 - i. no. (short for number - partial key)
 - ii. description
 - iii. Status (New, InProgress, OnHold, Blocked/Abandoned, Resolved)
 - iv. priority_no (decided by Manager who created the task, must be one of "Highest", "High", "Medium", "Low", and "Lowest")
 - v. creation_date
 - vi. closure_date (date and time the task ends, set by the Manager when the task is complete).

11. Meeting

- a. Scheduled events where at least 2 Employees (of any level) must join.
- b. Attributes:
 - i. meeting_id (unique identifier)
 - ii. title
 - iii. start-time (will be stored in date-time format, set by whoever schedules the meeting)
 - iv. end (expected end of the meeting by whoever scheduled the meeting)

b) Relationships captured and their real world meaning

- 1. Role specialization (isA)
 - a. Managers, Directors, Supervisors, and Standard employees are all Employees.
 - i. Covering: makes it so that each employee must exist with at least one role.
 - ii. Disjoint: enforces that an employee can only have one role at a time.
- 2. Belongs To (Standard - Team)
 - a. A Standard employee belongs to at most one Team.
 - b. A Team can have many Standard Employees.
- 3. Managed by (Team - Manager)
 - a. Each Team must be managed by a Manager.
 - b. A Manager can manage many Teams.
- 4. runs (Director - Project)
 - a. Each Director runs at most one Project.
 - b. Each Project can have many Directors, but needs at least one.
- 5. Comprises (Project - Components) (*Identifying relationship*)
 - a. A project comprises components.
 - b. A component can not exist without its project.
 - c. Component identity is (Project.codename, Component.name)
- 6. Subcomponent (Components - Components)
 - a. A component can have spinoff subcomponents (which can have their own spinoffs, etc.)
 - b. A subcomponent can only belong to one Component.
- 7. For (Objectives - Components) (*Identifying relationship*)
 - a. Each Objective belongs to and cannot exist without one Component.
 - b. Each Component can have many Objectives.
 - c. Objective identity is (Project.codename, Components.name, Objective.id).
- 8. For (Tasks - Objectives) (*Identifying relationship*)
 - a. Each Task belongs to and cannot exist without one Objective.
 - b. Each Objective can have many Tasks.
 - c. Task identity is (Project.codename, Components.name, Objective.id, Tasks.no).

9. Assigns (Manager - Standard - Task) (*Ternary relationship*)
 - a. A Manager assigns a Standard employee to a Task.
 - b. This captures the real-world fact that an assignment involves both the assigning manager and the assigned standard employee.
10. Attends (Employee - Meeting)
 - a. Employees may join Meetings.
 - b. Meetings may have many attending Employees of any subtype.

c) General constraints, restrictions, and special requirements

- **Keys and identification**

- Employee.emp_id is unique
- Team.team_id is unique
- Project.codename is unique
- Meeting.meeting_id is unique

- **Weak entity existence**

- Components.name is unique when combined with a project
- Objectives.obj_id is unique when combined with a component of a project
- Tasks.no is unique when combined with an objective of a component of a project

- **Role constraints (important)**

- Specialization Employee → {Standard, Manager, Supervisor, Director} is Covering and Disjoint (an Employee must have exactly one role at a time and an Employee can not exist without a role)
- Only a Manager can change the status of a Task that they assigned.
- A Manager can only change the status of Tasks that they created.
- Only a Manager can change the closure_date of a Task (done automatically in query when the Task status is changed to Resolved).
- Only a Manager can create a Task.
- A Manager can only create and assign tasks to Teams they manage.
- Only a Supervisor can create Objectives.
- A Supervisor can only create Objectives for Components they supervise, and subcomponents of components they supervise.
- A Supervisor can only assign Objectives to Teams where the Objective belongs to a Component they supervise, or a subcomponent of a Component that they supervise.
- Only Directors can create Components for a Project.
- Only Directors can create Subcomponents of a Component.
- Directors can only create Components for Projects they run.
- Directors can only create Subcomponents for Components that belong to Projects they run.

- Only Directors can create Projects.
- **Structural constraints**
 - Subcomponent relationships should be acyclic (a Subcomponent can't be a Subcomponent of itself directly or indirectly i.e a component can't be a subcomponent of one of its subcomponents anywhere down the hierarchy)
- **Temporal constraints**
 - `Project.start_date < Project.end_date` if `end_date` is not NULL
 - `Meeting.start_time < Meeting.end`
 - `Tasks.closure_date` is NULL by default.
- **Domain constraints**
 - `Task.priority_no` must be one of {"Highest", "High", "Medium", "Low", or "Lowest"}.
 - `Task.status` must be one of {"TODO", "IN_PROGRESS", "ON_HOLD", "DONE", "ABANDONED"}.

d) Functional Requirements (tasks or operations that can be performed on the data)

- 1) Create, update, and delete Employee records.
- 2) Classify each Employee into one role (Standard, Manager, Supervisor, Director).
- 3) Create, update, and maintain Teams.
- 4) Assign a Manager to each Team. Update team management when needed.
- 5) Assign Standard employees to each Team. Update team membership when needed.
- 6) Create, update, and maintain projects, including charters and dates.
- 7) Assign a Director to run each project. Update this assignment if needed.
 - a) Important note: There will be a default Director account seeded in. It will have APIs exposed to it to create new Director accounts.
- 8) Add Components to Projects.
- 9) Create Subcomponents.
- 10) Create, update, and delete Objectives and associate them with Components.
- 11) Assign Supervisors to supervise Components and distribute Objectives to Teams.
- 12) Create Meetings, update times, and record Employee attendance.
- 13) Assign Tasks via ternary relationship: (Manager assigns a Task to a Standard Employee).
- 14) Create, update, and delete Components.
- 15) Create, update, and delete Tasks under Objectives.
 - a) Update Tasks status/priorities/description.
- 16) Query operational views such as:
 - a) Tasks assigned to a given Standard employee.

- b) Tasks assigned by a given Manager.
 - c) Objectives under a given Component.
 - d) Components supervised by a given Supervisor.
 - e) Projects run by a given Director.
 - f) Attending Employees for a given Meeting.
 - g) Meeting attendance for a given Employee.
 - h) Component structure of a Project.
 - i) Hierarchical decompositions of Components and Subcomponents.
- Translation of the ER diagram to relations
 - Entity Sets:
 - **Employee:**
employee(emp_id, name NOT NULL, team_id, position_title NOT NULL, start_date NOT NULL, email UNIQUE NOT NULL, password NOT NULL), team_id REFERENCES Team
 - **Standard:**
standard(emp_id, name NOT NULL, position_title NOT NULL, start_date NOT NULL, email UNIQUE NOT NULL, password NOT NULL)
 - **Team:**
Team(team_id, Department NOT NULL, emp_id NOT NULL), emp_id REFERENCES manager
 - **Manager:**
manager(emp_id, name NOT NULL, position_title NOT NULL, start_date NOT NULL, email UNIQUE NOT NULL, password NOT NULL)
 - **Supervisor:**
supervisor(emp_id, name NOT NULL, position_title NOT NULL, start_date NOT NULL, email UNIQUE NOT NULL, password NOT NULL), name REFERENCES components
 - **Director:**
director(emp_id, name NOT NULL, codename, position_title NOT NULL, start_date NOT NULL, email UNIQUE NOT NULL, password NOT NULL), codename REFERENCES project
 - **Meeting:**
meeting(meeting_id, title NOT NULL, start_time NOT NULL, description, end_time NOT NULL, emp_id NOT NULL), emp_id REFERENCES employee
 - **Project:**
project(codename, charter, start_date NOT NULL, deadline_date NOT NULL)

- **Components:**

components(name, codename), codename REFERENCES project

- **Objectives:**

objectives(obj_id, name, codename, description, goal_name NOT NULL),
name REFERENCES component, codename REFERENCES project

- **Tasks:**

tasks(number, obj_id, name, codename, status UNIQUE NOT NULL,
description, priority_number), obj_id REFERENCES objectives,
codename REFERENCES project, name REFERENCES component)

- Relations:

- **Join:**

join(emp_id, title), emp_id REFERENCES employee, title
REFERENCES meeting

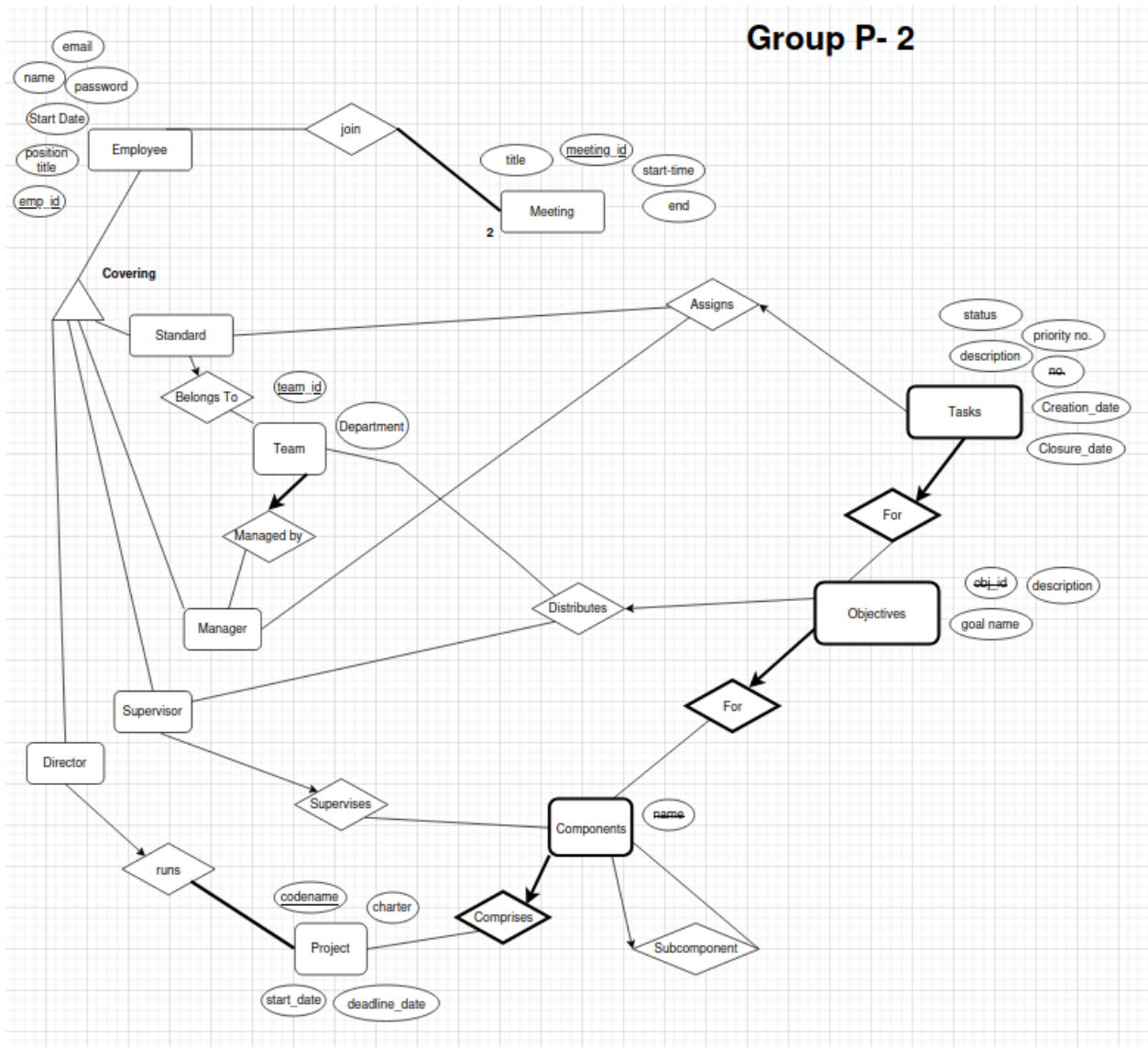
- **Assigns:**

assigns(standard_id, manager_id, tasks_no), standard_id REFERENCES
standard, manager_id REFERENCES manager, tasks_no REFERENCES
tasks

- **Distributes:**

distributes(obj_id, emp_id, team_id), obj_id REFERENCES objectives,
emp_id REFERENCES supervisor, team_id REFERENCES teams

Entity Relationship Model



Note: This ER Diagram is also attached as a separate PNG in our submission dropbox

How our Application Domain Stands Out

Our application is a meeting-driven task and project management system for organizations where accountability matters. Work items are traceable top-down starting from Projects to Components to Objectives and down to Tasks, while responsibility is enforced through the organisation structure (Directors run projects, Supervisors distribute objectives, Managers assign tasks, and Standard employees execute tasks). In contrast to many flat CRUD apps, this domain models real coordination, meetings are explicit artifacts with recorded participation, and work decomposition reflects how teams in the real world plan, manage, and deliver tasks.

Appropriate/Justifiable Modeling Concepts

Employee ISA hierarchy with a covering constraint. Employees are specialized into Standard, Manager, Supervisor, and Director with a covering constraint, so every employee participates in exactly one role due to the ISA hierarchy being non-overlapping by default (forcing each employee to belong to exactly one subclass). This is appropriate because role membership determines which relationships an employee may participate in (e.g., only Managers assign tasks, only Directors run projects).

Weak entities with identifying relationships: Components are identified within a Project (a component name can repeat across different projects), and Tasks are identified within an Objective (task numbers are meaningful within an objective). This matches the core motivation for weak entities, that the entity is existence-dependent and requires the owner's key plus a partial key to be uniquely identified.

Ternary relationship for task assignment and objective distribution: Task assignment records not just the Task and the assignee, but also the assigning Manager. This captures 'who assigned whom to what' and models the simultaneous interaction between the 3 entities, which is exactly when ternary relationships are warranted. The same is true for when supervisors assign objectives to teams.

Recursive relationships. Components can be decomposed into subcomponents using a recursive relationship. This supports realistic work breakdown structures.

Well-Defined Requirements and Breadth of ER Aspects

The requirement analysis is intentionally explicit about data semantics and integrity rules including information about role-based responsibilities, hierarchical work decomposition, and lifecycle details for tasks and projects. As a result, the ER model explores many aspects beyond basic entities and binary relationships, such as ISA hierarchies, weak entities with their identifying relationships, ternary relationships, and recursive structures. This breadth is achieved without forcing artificial constructs as each modeling choice follows directly from the domain's real world use cases.

Reasoning of Key Constraints

Key constraints are used only where they represent stable business invariants and where enforcing them improves integrity. These constraints translate cleanly into primary keys and foreign keys in the relational schema, supporting referential integrity.

Director/Project-Key Constraint on Runs Relationship. Each director can run at most one project, supporting clear ownership and an escalation project pathway if needed.

Supervisor/Component-Key Constraint on Supervisors Relationship. Each supervisor supervises at most one component, supporting clear ownership and an escalation component pathway if needed.

Team/Manager-Key Constraint on Managed By Relationship. Each team has exactly one responsible manager, a manager may manage multiple teams.

Standard/Team-Key Constraint on Belongs To Relationship. Each standard employee belongs to at most one team at a time which enables unambiguous reporting and allows for clear work distribution.

Weak Entity Key Constraints. Component identity depends on its Project and Task identity depends on its Objective. This prevents orphan work items and eliminates the need for artificial keys to identify these entities. In both cases their relationship sets must enforce a many to one constraint where each weak entity instance (ex. Components) is associated with exactly one owner instance (i.e. Project).

Inspirational Websites

Jira - <https://www.atlassian.com/software/jira>

MS Teams - <https://www.microsoft.com/en-ca/microsoft-teams/log-in>

Teamwork

The team had several meetings, approximately 10, throughout the course of the project. The meetings and associated work were roughly divided as such. 3 meetings between Alex, David, Brian, and Maxim for determining the application idea and validating its scope and feasibility with the Professor. 4 meetings between Alex, David, Brian, and Maxim, for constructing, editing, and finalising the ER schema after Alex's initial rough draft. 1 Meeting between Alex, David, Brian, and Maxim for editing and finalising the ER schema translation to its relational model after Maxim's initial draft. 1 Meeting between Alex, David, Brian, and Maxim for editing and finalising the application's requirement analysis after David's initial draft. 1 Meeting between Alex, David, Brian, and Maxim for editing and finalising the application's creativity and complexity description after Brian's initial draft.